23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

PROJECT REPORT

Group No: A9

Team Members:

Roll No	Student Name
AM.SC.U4CSE25018	Aditya Sahu
AM.SC.U4CSE25058	V Chetan Reddy
AM.SC.U4CSE25074	Syed Zaheer Ahmed
AM.SC.U4CSE25041	M Bhargav

Project Title: Course Feedback and Faculty Evaluation System

CONTENTS

Sl No	Chapter	Page No
1	Abstract	
2	Introduction	
3	Requirements	
4	UI Design	
5	Database Design	
6	Database Selection and Justification	
7	Implementation	
8	Testing and Validation	
9	Results and Screenshots	
10	Challenges Faced and Solutions	
11	Conclusion and Future Enhancements	
12	References	

1 Abstract

The Course Feedback and Faculty Evaluation System addresses the difficulty of collecting, storing, and analysing student feedback for courses and faculty members using manual forms or disconnected online forms. The objective of the project is to provide a secure database-backed application where students submit feedback only for enrolled courses, administrators manage academic master data, and faculty or administrators view meaningful aggregate reports. The proposed solution uses PostgreSQL for normalized relational storage, Node.js and Express for the backend API, and a plain HTML, CSS, and JavaScript frontend for a lightweight user interface. Major database features include primary and foreign key constraints, unique constraints to prevent duplicate enrolment and duplicate evaluation, check constraints for valid IDs and scores, password hashing using pgcrypto, deferred constraints for the department HOD relationship, triggers for score validation, and report queries for course, faculty, department, and student feedback summaries. The expected outcome is a reliable feedback system that improves data integrity, reduces repeated manual work, protects student confidentiality in reports, and supports accurate decision making for academic improvement.

2 Introduction

Course feedback is an important academic process because it helps departments understand teaching quality, course organization, laboratory effectiveness, student satisfaction, and areas that need improvement. In many colleges, feedback is collected through paper forms or simple online forms. These approaches are easy to start but difficult to maintain because staff must manually check duplicate responses, consolidate scores, and prepare reports. They also make it difficult to ensure that only enrolled students submit feedback for a specific course offering.

The proposed system focuses on the course feedback workflow used in a B.Tech CSE academic environment. Students log in using their institutional student ID and name, select one of their enrolled courses, answer a fixed set of evaluation questions on a 1 to 5 scale, and submit an optional comment. Administrators log in with an email and password, maintain student, faculty, and course data, and view dashboards and reports. Faculty members are represented in the database design so that individual faculty reports can be generated without exposing student identities.

The existing scenario has several limitations: feedback can be duplicated, unauthorised students can rate courses they did not study, reports are time consuming to prepare, and historical course offerings can be mixed up when the same course is taught by different faculty members in different semesters. The proposed database application addresses these issues by separating courses from teaching assignments, linking students through enrolment, storing one evaluation per enrolment, and storing question-wise feedback responses in a normalized structure.

3 Requirements

3.1 Functional Requirements

Requirement	Description
Authentication	Students log in with Student ID and name; admins log in with email and password hash verification.
Student Feedback Submission	Students select an enrolled course, answer active evaluation questions, and submit an optional comment.
Master Data Management	Admins add and update student, faculty, and course records.
Teaching Assignment Setup	The database stores course, faculty, academic year, term, semester, and section for each actual course offering.
Enrolment Control	Only students enrolled in a teaching assignment can submit feedback for that course offering.
Duplicate Prevention	A student can enrol only once in the same assignment and can submit only one evaluation for one enrolment.
Report Generation	The system supports course averages, question-wise averages, faculty reports, department summaries, recent feedback, and pending feedback lists.
Seed and Reset Support	Demo data can be loaded using seed.sql and reset through a backend API endpoint.

3.2 Non Functional Requirements

Quality	Requirement
Performance	Indexes on foreign keys support joins used by dashboards and reports.
Security	Passwords are stored as hashes using PostgreSQL pgcrypto; reports for faculty avoid student identity.
Reliability	Transactions are used for feedback submission so evaluation and response rows are saved atomically.
Usability	The frontend is plain HTML, CSS, and JavaScript with simple role-based navigation.
Scalability	The schema separates master data, assignments, enrolments, evaluations, and responses so new terms and questions can be added without redesign.
Maintainability	UI logic is separated from API/data logic, and SQL scripts are separated into schema, seed, reports, and tests.

4 UI Design

The user interface is named Feedback Hub and is implemented with vanilla HTML, CSS, and JavaScript. The same frontend can run as a local prototype or connect to the Express API. The interface has two main roles: student and admin.

Screen	Purpose	Main User Interaction

Login Screen	Allows role selection between Student and Admin.	Students enter Student ID and full name; admins enter email and password.
Student Feedback Form	Collects course feedback.	Student selects a course, answers evaluation sliders, writes an optional comment, and submits.
My Feedbacks	Shows feedback submitted by the logged-in student.	Student reviews submitted course, average score, date, and comment.
Profile	Displays student details.	Student views ID, department, semester, and feedback count.
Admin Dashboard	Shows summary metrics and course rating chart.	Admin scans totals, average rating, recent feedback, and course performance.
Students, Faculty, Courses	Provides add and update forms.	Admin selects rows, loads existing details, edits fields, and saves changes.
Feedback and Reports	Shows feedback summaries and course reports.	Admin refreshes results, exports feedback, and generates question-wise course reports.

Navigation uses a sidebar in both portals. Student users move between Submit Feedback, My Feedbacks, and Profile. Admin users move between Dashboard, Students, Faculty, Courses, Feedback, and Reports. The frontend calls the backend API when available and falls back to local demo data when the server is offline.

5 Database Design

5.1 ER Diagram

The ER diagram below shows the main entities and relationships. Department is the organizational root; teaching assignment represents an actual course offering; enrolment connects students to offerings; evaluation and feedback store one submitted form and its question-wise responses.

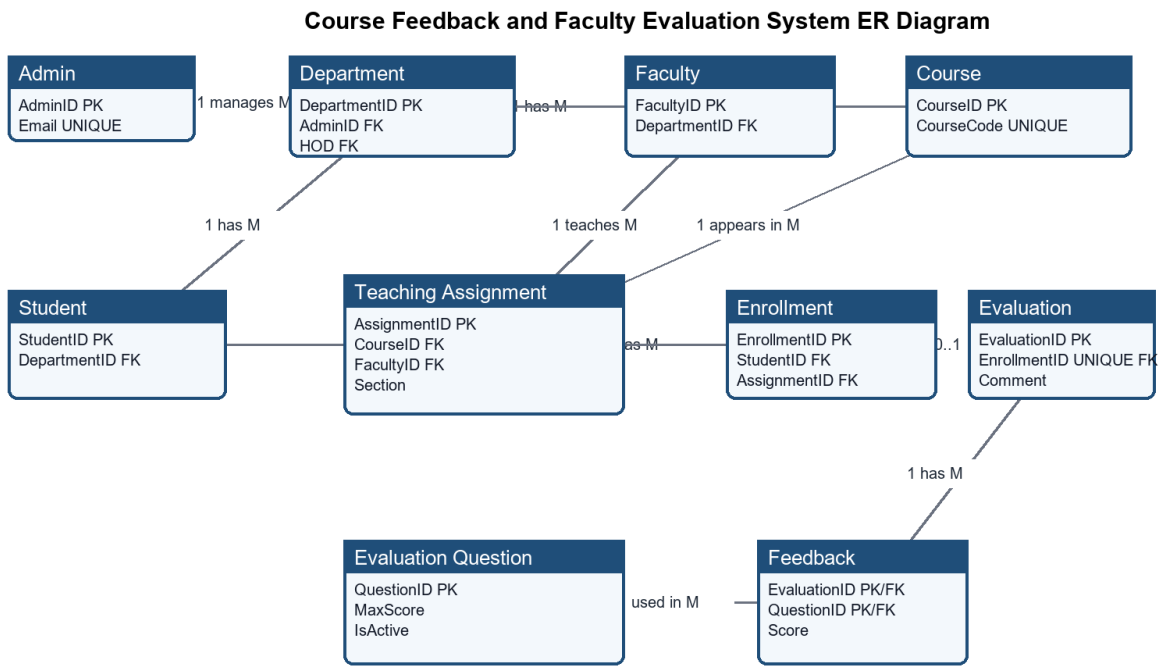


Figure 1. ER diagram for the Course Feedback and Faculty Evaluation System.

5.2 Entities and Attributes

Entity	Important Attributes
Admin	AdminID, name fields, email, hashed_password, created_at
Department	DepartmentID, DepartmentName, AdminID, HODFacultyID
Student	StudentID, name fields, email, hashed_password, semester, DepartmentID
Faculty	FacultyID, name fields, email, hashed_password, designation, DepartmentID
Course	CourseID, CourseCode, CourseName, SubjectLabel, Credits, DepartmentID
Teaching Assignment	AssignmentID, CourseID, FacultyID, semester, academic_year, term, section
Enrollment	EnrollmentID, StudentID, AssignmentID, EnrollmentDate
Evaluation Question	QuestionID, QuestionText, MaxScore, IsActive, DisplayOrder
Evaluation	EvaluationID, EnrollmentID, SubmissionDate, Status, Comment
Feedback	EvaluationID, QuestionID, Score, Remark

5.3 Relationships and Cardinalities

Relationship	Cardinality	Implementation
Admin manages Department	One admin can manage many departments.	department.admin_id references admin.admin_id.
Department has Students, Faculty, Courses	One department can have many child records.	Foreign keys in student, faculty, and course.

Department has HOD	Each department has one HOD; one faculty member heads at most one department.	department.hod_faculty_id is unique and uses a deferred same-department foreign key.
Course and Faculty form Teaching Assignment	One course and one faculty member create one offering.	teaching_assignment stores course, faculty, year, term, semester, and section.
Student enrolls in Teaching Assignment	Many students can join many assignments through enrollment.	enrollment resolves the many-to-many relationship.
Enrollment has Evaluation	At most one evaluation exists for each enrollment.	evaluation.enrollment_id is unique.
Evaluation answers Questions	One evaluation contains one response per question.	feedback has composite primary key on evaluation_id and question_id.

5.4 Relational Schema

```
Admin(AdminID, FirstName, MiddleName, LastName, Email, HashedPassword, CreatedAt)
Admin_Phone(AdminID, Phone)
Department(DepartmentID, DepartmentName, AdminID, HODFacultyID)
Student(StudentID, FirstName, MiddleName, LastName, Email, HashedPassword,
Semester, DepartmentID)
Student_Phone(StudentID, Phone)
Faculty(FacultyID, FirstName, MiddleName, LastName, Email, HashedPassword,
Designation, DepartmentID)
Faculty_Phone(FacultyID, Phone)
Course(CourseID, CourseCode, CourseName, SubjectLabel, Credits, DepartmentID)
Teaching_Assignment(AssignmentID, CourseID, FacultyID, Semester, AcademicYear,
Term, Section)
Enrollment(EnrollmentID, StudentID, AssignmentID, EnrollmentDate)
Evaluation_Question(QuestionID, QuestionText, MaxScore, IsActive, DisplayOrder)
Evaluation(EvaluationID, EnrollmentID, SubmissionDate, Status, Comment)
Feedback(EvaluationID, QuestionID, Score, Remark)
```

5.5 Constraints

The schema includes primary keys for all main tables, foreign keys for referential integrity, unique constraints on candidate keys such as email and course code, check constraints for semester, score, course code format, student ID format, and trigger validation to ensure feedback scores do not exceed the selected question's maximum score. Foreign-key indexes are created for efficient joins and delete checks.

5.6 Functional Dependencies and Normalization

Relation	Main Functional Dependencies
Admin	AdminID -> name, email, hashed_password; Email -> AdminID
Department	DepartmentID -> DepartmentName, AdminID, HODFacultyID; HODFacultyID -> DepartmentID
Student	StudentID -> name, email, semester, DepartmentID; Email -> StudentID
Faculty	FacultyID -> name, email, designation, DepartmentID; Email -> FacultyID
Course	CourseID -> CourseCode, CourseName, Credits, DepartmentID; CourseCode -> CourseID

Teaching Assignment	AssignmentID -> CourseID, FacultyID, AcademicYear, Term, Section
Enrollment	EnrollmentID -> StudentID, AssignmentID, EnrollmentDate; StudentID + AssignmentID -> EnrollmentID
Evaluation	EvaluationID -> EnrollmentID, SubmissionDate, Status, Comment; EnrollmentID -> EvaluationID
Feedback	EvaluationID + QuestionID -> Score, Remark

The design satisfies 1NF because all fields are atomic and repeating question scores are separated into the Feedback table. It satisfies 2NF because non-key attributes depend on full primary keys; for example, question text is stored in Evaluation_Question rather than inside Feedback. It satisfies 3NF because department, course, faculty, student, and assignment details are not copied into feedback rows. Candidate keys such as email, course code, enrolment pair, and evaluation enrolment are protected using unique constraints, supporting BCNF-level reasoning for the core relations.

6 Database Selection and Justification

PostgreSQL was selected as the DBMS because it provides strong SQL support, reliable transactions, rich constraint handling, triggers, extensions, indexing, and good support for relational database design. The pgcrypto extension supports password hashing inside the database. PostgreSQL also supports deferrable foreign key constraints, which are useful for the circular Department-HOD-Faculty relationship. It is open source, widely documented, and suitable for student projects as well as production applications.

Factor	Justification
Performance	Foreign-key indexes and SQL aggregates support dashboard and report queries efficiently.
Security	Password hashes can be verified using pgcrypto; row-level reporting can avoid exposing student identities.
Integrity	Primary keys, foreign keys, unique constraints, check constraints, and triggers enforce business rules.
Scalability	The normalized schema supports new departments, courses, terms, sections, and questions.
Cost	PostgreSQL is free and open source.
Ease of Use	It works well with pgAdmin, psql, Node.js, and the pg library.

7 Implementation

The implementation uses PostgreSQL for data storage, Node.js with Express for the backend API, and plain HTML, CSS, and JavaScript for the frontend. The database scripts are separated by purpose: schema.sql creates tables and constraints, seed.sql loads demo data, reports.sql contains report queries, and test_cases.sql validates constraints.

7.1 Project Files

File	Purpose
schema.sql	Creates schema, keys, constraints, indexes, triggers, and pgcrypto extension.
seed.sql	Loads CSE department, admin, faculty, students, courses, assignments, enrollments, questions, evaluations, and feedback.
server.js	Express backend with PostgreSQL connection pool and API routes.
data.js	Frontend data adapter that calls API endpoints with local fallback data.
index.html, style.css, script.js	Student and admin UI implementation.
reports.sql	Reusable SQL report queries.
test_cases.sql	PASS/FAIL validation test script.

7.2 Database Creation Commands

```
psql -U postgres -d postgres -c "CREATE DATABASE feedback_hub;"
psql -U postgres -d feedback_hub -f schema.sql
psql -U postgres -d feedback_hub -f seed.sql
```

7.3 Important SQL Snippets

The following examples show important implementation logic from the schema.

```
ALTER TABLE department
    ADD CONSTRAINT fk_department_hod_same_department
    FOREIGN KEY (department_id, hod_faculty_id)
    REFERENCES faculty(department_id, faculty_id)
    DEFERRABLE INITIALLY DEFERRED;
CREATE TRIGGER trg_feedback_validate_score
BEFORE INSERT OR UPDATE OF question_id, score ON feedback
FOR EACH ROW
EXECUTE FUNCTION validate_feedback_score();
```

7.4 API Routes

Endpoint	Method	Purpose
/api/health	GET	Checks database connectivity.
/api/data	GET	Loads admin, questions, students, faculty, courses, and feedback for the UI.
/api/login	POST	Authenticates student or admin.
/api/feedback	POST	Creates evaluation and question-wise feedback rows in one transaction.
/api/students	POST	Adds a new student.
/api/students/:id	PUT	Updates student details.
/api/teachers	POST	Adds a faculty member.
/api/courses	POST	Adds course and teaching assignment.
/api/reset-seed	POST	Reloads demo seed data.

8 Testing and Validation

Testing was planned around both normal functional behaviour and exceptional constraint cases. The test_cases.sql script catches expected failures and prints a PASS or FAIL result table.

Test Case	Input	Expected Output	Actual Output	Result
Admin login hash	admin@amrita.edu / admin123	One matching admin row	Password hash condition returns one row	Pass
Duplicate student	Insert existing AM.SC.U4CSE25018	Primary key violation	Unique violation raised	Pass
Duplicate enrollment	Same student and assignment twice	Unique constraint violation	Unique violation raised	Pass
Invalid student ID	BAD-ID	Check constraint violation	Check violation raised	Pass
Score above max	Score 6 for max score 5	Trigger rejection	Check violation raised	Pass
Inactive question	Submit response to inactive question	Trigger rejection	Check violation raised	Pass
Duplicate evaluation	Second evaluation for same enrollment	Unique constraint violation	Unique violation raised	Pass
Wrong HOD department	ECE department with CSE faculty HOD	Deferred foreign key rejection	Foreign key violation raised	Pass

The validation confirms that the database enforces entity integrity, referential integrity, domain constraints, duplicate prevention, password hash checking, and important cross-table business rules.

9 Results and Screenshots

The completed system supports student feedback submission, admin data management, and report generation using live PostgreSQL data. The seeded database contains 4 students, 3 faculty members, 3 courses, 5 evaluation questions, 4 evaluations, and 20 question-wise feedback rows. The frontend dashboard shows total counts, overall average rating, course-wise averages, and recent feedback comments.

Screenshot	Caption to Use in Final Submission
Login screen	Figure 2. Feedback Hub login screen with student and admin role selection.
Student feedback form	Figure 3. Student portal showing course selection, rating sliders, and comment box.
Admin dashboard	Figure 4. Admin dashboard showing student, faculty, course, and average rating metrics.
Course management	Figure 5. Admin course management screen for adding and updating course records.
pgAdmin tables	Figure 6. PostgreSQL public schema showing project tables.
SQL reports	Figure 7. Output of report query showing course-wise average ratings.

Test cases	Figure 8. PASS results from test_cases.sql validating database constraints.

Screenshots can be inserted at the positions listed above after running the application and SQL scripts on the local machine. Each screenshot should show the complete window area, have a short caption, and demonstrate either a UI workflow or a database result.

10 Challenges Faced and Solutions

Challenge	Solution
Circular Department-HOD-Faculty relationship	Created Department first and added a deferrable composite foreign key after Faculty creation.
Preventing duplicate feedback	Made evaluation.enrollment_id unique so each enrolment can have at most one submitted evaluation.
Ensuring only enrolled students submit feedback	Feedback submission API looks up an enrollment for the selected student and course before insertion.
Validating score against question maximum	Added a trigger that checks score against Evaluation_Question.max_score and rejects inactive questions.
Plain UI to real backend migration	Kept UI logic separate from data access in data.js and swapped mock calls for API calls.
Password handling	Used pgcrypto hashes instead of storing plaintext passwords.
Report privacy	Faculty report query uses anonymous aggregate scores and does not select student identity.

11 Conclusion and Future Enhancements

The project successfully implements a database-backed Course Feedback and Faculty Evaluation System. The schema models departments, students, faculty, courses, teaching assignments, enrollments, evaluation questions, evaluations, and feedback responses in a normalized form. PostgreSQL constraints and triggers enforce important rules such as valid student IDs, valid scores, one evaluation per enrollment, and same-department HOD assignment. The Node.js backend connects the database to a plain HTML, CSS, and JavaScript UI, allowing students to submit feedback and admins to manage data and generate reports.

Future enhancements can include JWT-based sessions, separate faculty login screens, role-based authorization middleware, CSV or PDF export of reports, richer analytics dashboards, mobile-responsive improvements, email notifications for pending feedback, cloud deployment, and row-level security for stronger confidentiality. Additional report views can also compare trends across academic years, sections, and departments.

12 References

1. PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/

2. Express.js. Express Web Framework Documentation. https://expressjs.com/

3. node-postgres. PostgreSQL Client for Node.js Documentation. https://node-postgres.com/

4. Amrita School of Computing. 23CSE202 Database Management Systems Project Report Format.

5. Course Feedback and Faculty Evaluation System project source files: schema.sql, seed.sql, server.js, data.js, reports.sql, and test_cases.sql.